Before we design tasks that require the aforementioned leap of thought, we first clarify what tasks do not require it.

Open-ended tasks that do *not* require a leap of thought. One simple open-ended task that may come to mind is generating uniformly-random known entities, like celebrity names (Zhang et al., 2024b). However, there is no opportunity to create a novel string here. A more interesting example may be generating grammatically coherent probabilistic CFG strings following a `subject verb object` format e.g., `the cat chased a rat` (Hopkins et al., 2023). While novel strings become possible now, no sophisticated leaps of thought are involved; each token can be generated on the fly, satisfying a local next-token constraint to be coherent. In light of this, we can rephrase our goal as designing open-ended, creative tasks where coherence requires satisfying more interesting, "global" constraints. To build this systematically, we draw inspiration from literature in cognitive science (Boden, 2003). Boden (2003) argues that fundamentally, there are three forms of creativity in that order: *combinational, exploratory* and *transformative*. We elaborate on the first two (the last, we do not look at).

2.1. The fundamental types of creativity (Boden, 2003)

Combinational creativity. Consider rudimentary wordplay of the form "`What musical genre do balloons enjoy? Pop music.`" or "`What kind of shoes do spies wear? Sneakers.`" There is a global structure here: two unrelated entities (`genre` & `balloons`) are related eventually through a punchline (`pop`); the punchline itself is a mutual neighbor on a semantic graph. More broadly, Boden (2003) argues that many tasks, like the above, involve "making unfamiliar combinations of familiar ideas" or the "unexpected juxtaposition of [known] ideas". Other tasks include drawing analogies, or finding connections between disparate ideas in science. All these tasks involve a leap of thought that in effect searches and plans over a space of known facts and combines them.

Exploratory creativity. Consider the act of developing a mystery or designing logical puzzles. These endeavors are not as knowledge-heavy. What they crucially require is constructing fresh patterns that satisfy some highly non-trivial global constraint e.g., being resolvable as per some rules (e.g., logic). Such endeavors fall into a second class of *exploratory* creativity in Boden (2003). This includes much grander forms of exploration e.g., exploring various forms of outputs within a stylistic constraint, or exploring various corollaries within a theoretical paradigm in physics or chemistry. The leap of thought here requires searching over all possible sequences, constrained by a set of rules.

In the upcoming sections, we will capture some core computations of rudimentary instances within the two creative skills above. By no means does this minimal algorithmic setup intend to capture the human values or the subjective aspects of these endeavors (Schmidhuber, 2009); nor do they capture the rich array of creative acts that Boden (2003) discusses within these categories. (See limitations in §6).

2.2. The basic setting and notations

In all our tasks, we assume the standard generative model setting: the model must learn an underlying distribution $\mathcal{D}$ through a training set S of m independent samples $\boldsymbol{s}_i \sim \mathcal{D}$. The distribution is over a space $\mathbb{V}^L$ of L-length strings composed of tokens from the vocabulary $\mathbb{V}$. The tasks are open-ended in that there is no one correct answer at test-time. The goal *is* to produce a random string from $\mathcal{D}$, much like responding to the query `Design a high-school word problem`. We hope that this setup loosely resembles pretraining for language or in applications like drug discovery or protein (Meier et al., 2021; Madani et al., 2023; Watson et al., 2023) and genome modeling (Rives et al., 2021; Nguyen et al., 2024), where the model is trained on a set of varied example sequences, and is later used to produce novel outputs during inference.

Coherence: Each task is defined by a boolean coherence function $\texttt{coh} : \mathbb{V}^L \mapsto \{\texttt{true}, \texttt{false}\}$ which is true only on the support i.e., $\texttt{supp}(\mathcal{D}) = \{\boldsymbol{s} \in \mathbb{V}^L | \ \texttt{coh}(\boldsymbol{s})\}$. The exact form of `coh` will be defined in each algorithmic task but broadly, we are interested in scenarios where determining coherence requires a global understanding of the whole string. This is inspired by the fact that a wordplay must have a preplanned punchline connecting what comes before, or a word problem must be solvable. We can think of $\mathcal{D}$ to be a simple uniform distribution over all coherent strings.

Algorithmic creativity: Upon witnessing a finite set of examples, the model must learn to generate only strings that are (a) coherent, (b) original (not memorized) and (c) diverse (covers the whole support). An exact quantification of this is computationally expensive in our tasks. Instead, we approximate it by sampling a set T of many independent generations from the model and computing the fraction of T that is original, coherent and unique. Let the boolean $\texttt{mem}_S(\boldsymbol{s})$ denote whether an example $\boldsymbol{s}$ is from the training set S and let the integer function $\texttt{uniq}(X)$ denote the number of unique examples in a set X. (The exact definitions of these quantities vary by tasks, as we will see). Then, we define our (empirical) algorithmic creativity metric:

$$\hat{\texttt{cr}}_N(T) = \frac{\texttt{uniq}(\{\boldsymbol{s} \in T | \neg \texttt{mem}_S(\boldsymbol{s}) \wedge \texttt{coh}(\boldsymbol{s})\})}{|T|}. \quad (1)$$

Admittedly, we are looking at a simple form of novelty that is in-distribution. This notion, while simple, finds its relevance in applications like modeling proteins and genomes. We will also see that this notion is non-trivial enough to demarcate the limits of language models.

2.3. Tasks inspired by combinational creativity

Combinational creativity requires a recall-and-search through entities from memory, constrained to relate with each other in an interesting way. We abstract this through tasks that discover structures from an *in-weights* graph i.e., a graph stored in the model weights, not reveal in context.

2.3.1. Sibling discovery

This task involves an implicit, bipartite $\mathcal{G}$ with parent vertices $\mathcal{V} = \{A, B, C, \ldots\}$ each neighboring a set of children $\texttt{nbr}(A) = \{a_1, a_2, \ldots, \}$, $\texttt{nbr}(B) = \{b_1, b_2, \ldots\}$ and so on. We define $\texttt{coh}(\boldsymbol{s})$ to be true on sibling-parent triplets of the form $\boldsymbol{s} = (\gamma, \gamma', \Gamma)$ such that $\gamma, \gamma' \in \texttt{nbr}(\Gamma)$. We then consider a uniform distribution $\mathcal{D}$ over all coherent strings $(\gamma, \gamma', \Gamma)$ for a fixed graph $\mathcal{G}$. The model witnesses i.i.d samples from $\mathcal{D}$. During test-time, the model must maximize algorithmic creativity (Eq 1) by generating novel parent-sibling triplets based on its in-weights knowledge of $\mathcal{G}$. Note that the model is not provided the graph in-context as this would sidestep a core computational step in combinational creativity: recalling facts from a large memory (see §B.2). The hope is that the model infers and stores the pairwise adjacencies of $\mathcal{G}$ in its weights (given sufficient data). Full dataset description is in §C and Fig 9.

We view this task as an abstraction of the wordplay example. One can think of the parent Γ as the "punchline" that delivers a connection between otherwise non-adjacent vertices, in the same way `sneaker` surprisingly connects the otherwise non-adjacent words, `spies` and `shoes`.

What is a leap of thought? The natural order of generation is to pick the parent vertex (i.e., punchline) first, and the siblings after (conditioned on the parent). Thus, if the task demanded producing $(\Gamma, \gamma, \gamma')$, the tokens we observe betray the way they are naturally generated. However, the wordplay example presents the punchline (the parent, Γ) last while implicitly it must be planned ahead. We term this implicit (random) planning step a leap of thought. Paralleling this, we define our sibling discovery task to generate the triplets as $\boldsymbol{s} = (\gamma, \gamma', \Gamma)$, where the parent appears last. We hypothesize that this (sibling-first) construction is adversarial towards next-token learning, while a reversed (parent-first) dataset is friendlier towards next-token learning. (More on this in §2.6.)

2.3.2. Triangle discovery

Next, we design a task that requires a more complex, higher-order planning: generating triangles from an appropriately-constructed knowledge graph $\mathcal{G} = (V, E)$ (see graph construction in §C). Thus, in this task $\texttt{coh}((v_1, v_2, v_3)) = \texttt{true}$ iff all three edges between $\{v_1, v_2, v_3\}$ belong in $\mathcal{G}$. Furthermore, we define $\texttt{uniq}(\cdot)$ and $\texttt{mem}(\cdot)$ such that various permutations of the same triangle are counted as one (see details in §C, including the exact formatting of the string). Note that the leap of thought in this task is much harder to learn and execute as it requires co-ordinating three edges in parallel, from memory.

This type of a higher-order planning task can be thought of an abstraction of more complex wordplay (like *antanaclasis*, where a word must repeat in two different senses in a sentence, while still being coherently related to the rest of the sentence) or creating word games (like crosswords) or discovering contradictions or feedback loops in a body of knowledge, an essential research skill — see §B.3.

2.4. Tasks inspired by exploratory creativity

We are also interested in the less-knowledge-intensive creativity involved in tasks like designing word problems that demand novel solutions. We capture this skill minimally through tasks that construct new structures. No knowledge graph is involved in these tasks.

2.4.1. Circle construction

In this task, the model must generate adjacency lists that can be rearranged to recover circle graphs of N vertices. Let the generated list be $\boldsymbol{s} = (v_{i_1}, v_{i_2}), (v_{i_3}, v_{i_4}), \ldots$. We define $\texttt{coh}(\boldsymbol{s}) = \texttt{true}$ iff there exists a *resolving* permutation π such that $\pi(\boldsymbol{s}) = (v_{j_1}, v_{j_2}), (v_{j_2}, v_{j_3}), \ldots (v_{j_n}, v_{j_1})$ for distinct $j_1, j_2, \ldots j_n$. i.e., each edge leads to the next, and eventually circles back to the first vertex. We define `uniq` and `mem` such that different examples with the same resolving π are counted as the same, even if they have differing vertices. As always, the learner is then exposed to a finite set of uniformly sampled coherent strings. Note that the latent leap of thought here requires constructing a novel permutation π before generating the sequence.

Loosely, we can think of the resolving permutation π as *how* a conflict in a story or a word problem or a puzzle is solved; the vertices as characters or mathematical objects; the rules of rearranging an adjacency list as rules of logic, math or story-building. The creative goal in this task is to create novel dynamics in the conflict, or equivalently, novel dynamics in how the conflict is resolved. Thus if only the entities differ, but the plot dynamics remain unaltered, we count them as duplicates. See details in §C.

2.4.2. Line construction

A simple variant of the above task is one where the edge list is of a line graph. The resolving permutation π satisfies $\pi(\boldsymbol{s}) = (v_{j_1}, v_{j_2}), (v_{j_2}, v_{j_3}) \ldots, (v_{j_{n-1}}, v_{j_n})$ for distinct $j_1 \ldots j_n$. i.e., each edge leads to the next until a dead-end.

2.5. Leap of thought is obscured at the token-level

Rarely does human-written creative output come annotated with an articulation of the background creative process. Nor does each protein or molecule come with an enumeration of the laws that generated it — in fact, it is because those laws are not fully known do we delegate the task of inferring those to a machine. Likewise, in our setting, the training signal consists only of the observed tokens; the model must infer the underlying leap of thought.

In fact, in our last three tasks, the model must infer a deeper form of implicit structure than what exists in many typical algorithmic tasks, like addition (Lee et al., 2024), or the path-star task (B&N'24) or `Sibling Discovery`. Whereas in those tasks, the leap of thought is perceptible at the token-level, modulo some re-ordering, our last three tasks have no such token ordering. These task are permutation-invariant — no token is more privileged than the other, and all tokens must be "simultaneously learned" to infer the underlying process. We view this as an abstraction of tasks where the creative process is not immediate from the surface of the text. These tasks offer a test-bed even for non-next-token approaches that rely on re-permuting the tokens (Pannatier et al., 2024; Thankaraj et al., 2025) or predicting only a part of the future (Kitouni et al., 2024; Nolte et al., 2024; Bavarian et al., 2022; Fried et al., 2022).

2.6. How next-token learning may suffer in our tasks

Much like in sophisticated creative tasks, in our tasks, the most natural way to generate the string is by planning various random latent choices (say $\boldsymbol{z}$) in advance and by producing a *plan-conditioned* distribution $p(\boldsymbol{s}|\boldsymbol{z})$ over coherent strings $\boldsymbol{s}$. However, next-token prediction (`NTP`) – or next-token learning to be precise – we argue, is myopic and may struggle to learn such a latent plan. Our argument extends that of B&N'24 to our even simpler tasks.

Consider the sibling task where we must generate sibling-parent triplets $(\gamma, \gamma', \Gamma)$. The most natural generative rule is to plan the last token (the parent) first and decide the children last. Think of this as learning a latent plan $\boldsymbol{z} := \Gamma$. Then, learning the plan-conditioned generator $p(\gamma, \gamma', \Gamma|\boldsymbol{z})$ factorizes to learning the distribution of children conditioned on a parent as $p(\gamma|\boldsymbol{z} := \Gamma)$ and $p(\gamma'|\boldsymbol{z} := \Gamma)$ (due to conditional independence), and the trivial $p(\Gamma|\boldsymbol{z} := \Gamma)$. This requires only as many parent-sibling edges as there are in the graph, i.e., $O(m \cdot n)$ many points, if there are m parents, each with n children. This is optimal.

Things should proceed differently with `NTP`. We argue that a `NTP`-learner would fail to learn the plan $\boldsymbol{z} := \Gamma$. The key intuition is that an `NTP`-learner learns the parent Γ witnessing the siblings (γ, γ') as input. This is trivial to fit: the parent is simply the mutual neighbor of the two siblings revealed in the prefix; B&N'24 term this a "Clever Hans cheat" since the model witnesses and exploits part of the ground-truth it must generate (the `siblings`). Such cheats, being simpler than even the true generative rule, are quickly picked up during learning due to the well-known simplicity bias of gradient-descent-trained neural networks (Shah et al., 2020). Subsequently, any gradient supervision from the parent Γ is lost (also known as gradient starvation (Pezeshki et al., 2021)), leaving no guidance to learn the latent plan, $z := \Gamma$.

After the Clever Hans cheat is learned, we conjecture that the `NTP`-learner learns the second sibling not through the plan-conditioned distribution $p(\gamma'|\boldsymbol{z} := \Gamma)$ but through the next-token-conditional, $p(\gamma'|\gamma)$. This is a complex distribution: learning this may require witnessing every sibling-sibling pair totalling $O(m \cdot n^2)$ many training data — larger by a factor of n compared to the natural rule.

Summary of argument. Abstractly, in a creative leap-of-thought task, it is most efficient to learn a well-planned random latent $p(\boldsymbol{z})$ and a subsequent latent-conditioned distribution $p(\boldsymbol{s}|\boldsymbol{z})$. However, `NTP` factorizes this into pieces of the form $p(s_i|\boldsymbol{s}_{<i}, \boldsymbol{z})$. Consequently, on the later tokens, the model may be lured by Clever Hans cheats and learn uninformative latents. Conversely, the model may learn the earlier through complex rules, bereft of a latent plan. While this may not lead to complete breakdown of learning as in B&N'24 in our tasks, we hypothesize it must lead to data-hungry learning.

3. Training and Inference

Transformers. For our next-token-trained (`NTP`) models, we use the standard teacher-forcing objective used in supervised finetuning. Given prompt $\boldsymbol{p}$ and ground truth sequence $\boldsymbol{s}$, the model is trained to predict the i'th token s_i, given as input the prompt and all ground truth tokens up until that point, $(\boldsymbol{p}, \boldsymbol{s}_{<i})$. We write the objective more explicitly in §A Eq 2. For the multi-token Transformer models, we use teacherless training (Monea et al., 2023; Bachmann & Nagarajan, 2024; Tschannen et al., 2023), where the model is trained to predict s_i simultaneously for all i, only given the prompt $\boldsymbol{p}$ (and some dummy tokens masking the $\boldsymbol{s}$ that was once given as input). Since the exact details of this is irrelevant to our discussion, we describe this in Eq 2. To train our models, we use a hybrid of this objective and the next-token objective.

Diffusion models. We use the score entropy discrete diffusion model (`SEDD (90M)`, Lou et al., 2023). Loosely, we can think of the objective here as a hybrid of the teacherless objective, where the input is fully masked or corrupted (and the model must unmask or uncorrupt all of it), and various easier objectives with partial maskings or corruptions of the input. During inference, the model starts with a fully